Week 10 Lab Handout

Integration Week — Calling Your Partner's Live API

Time: 110 minutes

Bring: your laptop, your app and API from Weeks 5–9, your upstream partner's `openapi.yaml` and README, and the API key they sent you privately last week

---

Before you start: this is the week it all gets used

> For nine weeks you've built to a contract: designed it, implemented it, tested it, documented it, and locked it behind a key. Until today, nothing you built has done the job it was built for — one team's code calling another team's running API.

Expect things to break. That is not a sign you did something wrong; it's the point of the week. Almost every integration bug lives at the seam between two codebases, and the seam belongs to both teams. So this week's log records every bug **regardless of whose code was at fault** — a bug you find in your partner's API is as valuable as one in your own, and fixing it together is the skill being graded.

Part A — Preflight (10 min)

Before writing any code, confirm you have everything the first call needs:

1. **Your partner's base URL**, and confirmation their server is running right now. (If it runs on their laptop, they must keep it running and reachable from your machine for the whole lab.)
2. **Your key from them** — the one they issued to *you*, not your own team's key.
3. **Their contract and README**, open in a tab. The contract is your source of truth for field names, types, and status codes.
4. **A live channel** with your partner (chat or a shared call) for the whole lab. Agree who your contact is on each side.
5. **A check-in slot.** Your instructor or TA runs rotating 10-minute check-ins; sign up on the shared sheet now, not when you're already stuck.

Part B — Make the First Call Outside Your App (15 min)

Don't start inside your own code. Call the partner's API from a terminal first, so any problem is clearly theirs, the network's, or your key's — not your app's.

```
curl -i -H "X-API-Key: <the-key-they-gave-you>" https://partner-url/tasks
```

(Use the header name and path from *their* README — don't assume yours.)

Compare what comes back with their contract: status code, field names, types. Then **create `INTEGRATION_LOG.md` right now** and record this first call, whether it worked or not. The log is easiest to write as you go and nearly impossible to reconstruct at the end.

If this step fails, fix it before moving on. Use the debugging ladder at the end of Part C.

Part C — Wire It Into Your App (35 min)

Add one real feature to your app that fetches and displays data from your partner's API — a page, a panel, or a button that loads their data. Pick the smallest feature that is genuinely useful to your app (for example, a Dashboard showing upcoming tasks from the Task Tracker).

Call the partner from your server, not from the browser

If your frontend calls the partner directly, your partner's key sits in your page's JavaScript where anyone can read it — exactly the leak Week 9 taught you to avoid. Instead, your own backend makes the call, and your frontend asks your backend:

```
const PARTNER_URL = process.env.PARTNER_URL; // set in .env, never hardcoded
const PARTNER_KEY = process.env.PARTNER_API_KEY; // never hardcode, never log

app.get('/upcoming', async (req, res) => {
  try {
    const r = await fetch(`${PARTNER_URL}/tasks?status=active`, {
      headers: { 'X-API-Key': PARTNER_KEY },
    });
    if (!r.ok) {
      console.error('partner returned', r.status); // log the status, never the key
      return res.status(502).json({ message: 'Partner data unavailable' });
    }
    const tasks = await r.json();
    res.json(tasks.map(t => ({ title: t.title, dueDate: t.dueDate })));
  } catch (err) {
    console.error('partner call failed:', err.cause?.code || err.message);
    res.status(502).json({ message: 'Partner data unavailable' });
  }
});
```

(This is Node 18 or newer, which has `fetch` built in. In Flask, Django, or Laravel the shape is identical: make the request with the key header, check the status, read the JSON, return only what your page needs.)

Then your frontend calls your own route, with no key involved:

```
const res = await fetch('/upcoming');
if (!res.ok) { showMessage('Partner data is unavailable right now'); return; }
render(await res.json());
```

Rules for this step

- Put `PARTNER_URL` and `PARTNER_API_KEY` in `.env`, and add both names (with placeholder values) to `.env.example`. `.env` stays out of git.
- Use the **field names from their contract**, exactly. A field you spelled differently doesn't raise an error — it just comes back empty.
- Print the raw partner response once while you build, so you can see the real shape. Remove that print before you commit.

If you really do need to call the partner from the browser

Browsers check that the *other* server agrees to be called from your page, using CORS. A request carrying a custom header like `X-API-Key` triggers a **preflight**: the browser first sends an `OPTIONS` request, with no key, asking permission. The error you'll see in the console is "blocked by CORS policy".

The fix belongs to the **provider** (your partner), and the order matters:

```
const cors = require('cors');
app.use(cors({ origin: 'http://localhost:5173' })); // the consumer's exact origin
app.use(requireApiKey); // key check comes AFTER cors
```

If the key check runs first, the preflight gets a 401 with no CORS headers, and the browser reports a CORS problem even though the real cause is middleware order. Ask your partner for the change using a proper bug report (below). Don't guess their origin — give them yours, including the port.

The debugging ladder — use it whenever something fails

1. **Can you reach it at all?** `curl` the partner's URL. No answer means their server is down, the URL is wrong, or the network is blocking you — not your code.
2. **Does `curl` with the key work?** A 401 means the key or header name is wrong. A 429 means you've hit their rate limit; wait for the `Retry-After` time.
3. **What does your app actually send?** Log the URL and the header *names* (never the values).
4. **What came back?** Log the status and body in your terminal.
5. **Compare it to their contract.** Field names, types, nesting, status codes.
6. **Decide whose side the mismatch is on.** The contract says X and the API does Y: their bug. The contract is vague and you each read it differently: fix the contract together. You misread it: your fix. Whichever it is, log it.

Report bugs to your partner like an engineer

"It's broken" gives them nothing to act on. Send this instead:

```
What I did: GET /tasks?status=active with my key
What I expected: 200 and a list where each task has dueDate (your contract)
What I got: 200, but each task has due_date instead of dueDate
Evidence: (paste the real response, with the key removed)
When: 12:41, from my machine
```

Assume good faith: their API may be right and your reading wrong. Say what you saw, not what you think they did.

Part D — Handle One Failure Gracefully (15 min)

Your feature must survive your partner's API being unavailable. Today the bar is simple: no crash, no blank page — a clear message to the user.

1. With your feature working, ask your partner to **stop their server** (or point your `PARTNER_URL` at an address that doesn't exist).
2. Reload your page. You should see your friendly message, and your own server must still be running and answering its other routes.

3. Check your terminal. The log line should tell you *why* — `ECONNREFUSED` means nothing is listening at that address; `ENOTFOUND` means the hostname is wrong — so it's useful rather than just "failed".
4. Restart the partner and confirm your feature recovers without you changing code.

Do not build timeouts or retries today. A partner that is merely **slow** rather than down will make your page wait, and fixing that properly is next week's lesson.

Part E — Be a Good Provider (20 min)

While you consume your upstream partner, your **downstream** partner is consuming *you*. Their bugs are your problem too.

1. Keep your server running and watch your terminal. Look for 401s (their key or header is wrong), 404s (they're calling a path your contract doesn't have), and 429s (they are calling too often for your rate limit).
2. When they report something, work through the same debugging ladder from your side. Is your API doing what your own contract says?
3. If the bug is yours, fix it, tell them it's fixed, and **add it to your log** — it counts even though it was your code.
4. If your contract was ambiguous, clarify the contract so the next reader can't make the same mistake.

Part F — Finalize and Rehearse Your Demo (15 min)

1. **Finish `INTEGRATION_LOG.md`.** One row per bug or surprise:

#	What broke (symptom)	Root cause	Fix	Whose code, and who fixed it
1	Our page was empty and showed no error	Partner returned `due_date`, but their contract says `dueDate`	Partner fixed their response; we added a shape check	Partner, fixed by partner
2	Browser console: "blocked by CORS policy"	Key check ran before CORS, so the preflight got 401	Provider moved `cors()` above the key check	Provider, fixed by provider

(These two are examples — yours will differ. Don't invent bugs. If your first call went smoothly, say so, then show what you *tested on purpose*: a wrong key, an ID that doesn't exist, a field-shape check, the partner stopped. Record what each one did.)

2. **Rehearse the demo in three steps:** the page shows live partner data; the partner stops and your page shows its fallback message; the partner restarts and the data comes back.
3. Commit and push:

```
git add . INTEGRATION_LOG.md .env.example
git commit -m "Week 10: live integration with upstream partner"
git push
```

Before you push, run `git status` and confirm `.env` is **not** listed.

This week's deliverable checklist

- A feature in your app that calls and displays data from your partner's live API, working end to end
- The partner's key and URL come from environment variables; `.env` is not committed
- Field names match the partner's contract
- With the partner stopped, your app shows a clear message and does not crash
- `INTEGRATION_LOG.md` lists each bug or surprise with symptom, root cause, fix, and whose code it was
- You supported your downstream partner and logged anything that was your side's bug
- Demo rehearsed: live data, partner down, partner back
- All code committed and pushed

Rubric (30 pts)

Criterion	Points
Integration actually works end-to-end (live demo)	15
Integration handles at least one failure case gracefully (e.g., partner API down)	6
Integration log shows real debugging process, not just "it worked"	6
Evidence of constructive cross-team collaboration	3

Common problems and quick fixes

Problem	Fix
Every call returns 401	Check three things: the header name matches their README exactly, you're sending the key *they* issued you (not your own team's), and your `.env` loaded without a stray space or newline in the value
Browser says "blocked by CORS policy", but `curl` works	The browser sent a preflight with no key. Either call from your own server instead (recommended), or ask the provider to put `cors()` before their key check, allowing your exact origin
It works, then starts returning 429	You're calling on every page load or refresh. Their limit is per key — call less often, or keep the last result for a short time instead of refetching
The log says `ECONNREFUSED` or "fetch failed"	`ECONNREFUSED`: nothing is listening at that address, so the partner's server is off or the port is wrong. `ENOTFOUND`: the hostname is misspelled
You get a 200 but the page shows blanks or `undefined`	Field names differ from the contract. Print the raw response and compare it with their contract field by field
It works for you but your partner can't reach your server	Their machine can't see yours: check you're on the same network and your firewall allows it, or share a URL that is reachable from outside your laptop
Your partner's server isn't ready	Tell your instructor or TA at your check-in right away. Meanwhile build against a small stand-in that returns the example responses from their contract, and switch the URL once they're live. Log it either way

Next week: Resilience. Today you handled a partner that was down. Next you'll handle one that is slow, flaky, or wrong — timeouts, retries, and graceful degradation.